

TP1 Architecture and Detailed Design

CSE 360 Team Project, Phase 1 — Lessons Learned System (Foundation Code)

Role: Design Documents and Architecture — integration coordination and shared infrastructure
Author: A.G. (agupt545)
Version: 1.00
Date: 2026-09-17
Applies to: TP1/src, as built and verified by `testingAutomation.TP1RegressionSuite`

1. Purpose of this document

This document records the architecture of the TP1 application and the detailed design of the parts that changed during this phase. It exists so that the requirements, the architecture, the detailed design, the code, and the tests can be read against one another and kept in agreement. Its companion document, the *TP1 Traceability Matrix*, gives the line-by-line mapping from each user story to the code and the test cases that verify it.

The diagrams were produced from the PlantUML sources in the `UML Sources` folder so that any team member can regenerate them after a change. Those sources are to be recreated in Astah for TP2 and TP3, as the course requires; the shapes, names, and relationships are already in the form the Astah models will take.

2. Architectural view

The application is layered. Each layer depends only on the layers beneath it, which is what allows a feature owner to change one page without disturbing anyone else's work.

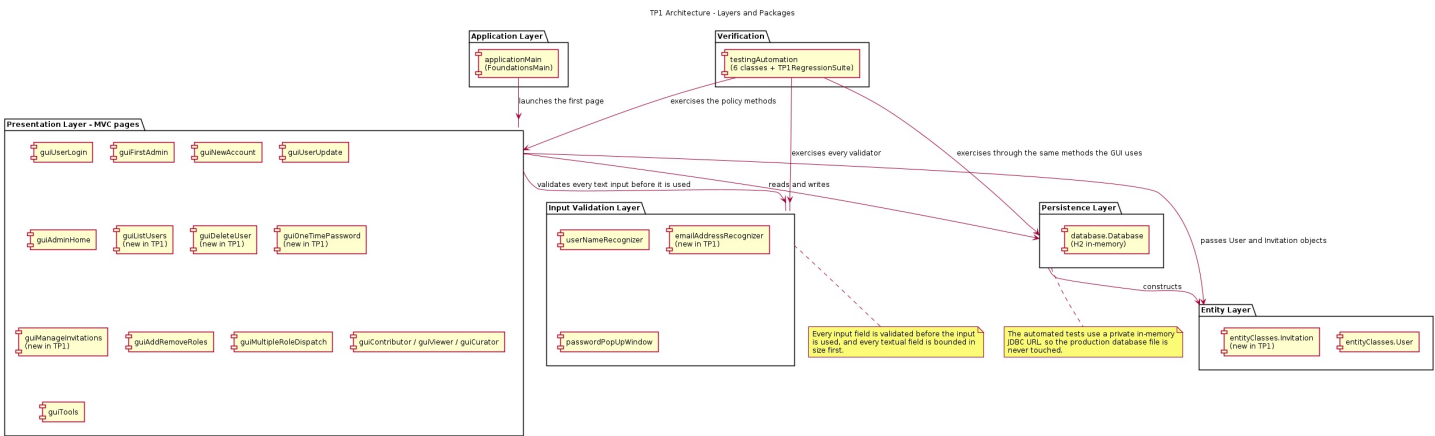


Figure 1 — TP1 architecture: layers and the packages within them. Packages marked “new in TP1” were added this phase.

2.1 The layers

Layer	Responsibility	Packages
Application	Starts JavaFX, connects to the database, and decides whether the first screen is the First Admin setup or the login page.	<code>applicationMain</code>
Presentation	One package per page, each a Model-View-Controller triple. The View owns the widgets, the Controller owns the actions, and the Model is a stub where the page holds no state of its own.	<code>guiUserLogin</code> , <code>guiFirstAdmin</code> , <code>guiNewAccount</code> , <code>guiUserUpdate</code> , <code>guiAdminHome</code> , <code>guiListUsers</code> , <code>guiDeleteUser</code> , <code>guiOneTimePassword</code> , <code>guiManageInvitations</code> , <code>guiAddRemoveRoles</code> , <code>guiMultipleRoleDispatch</code> , <code>guiContributor</code> , <code>guiViewer</code> , <code>guiCurator</code> , <code>guiTools</code>
Input validation	Pure functions that decide whether a piece of text may be used. They hold no state and touch no database, which is why the automated tests can call them directly.	<code>usernameRecognizer</code> , <code>emailAddressRecognizer</code> , <code>passwordPopUpWindow</code>
Entity	The objects passed between layers.	<code>entityClasses.User</code> , <code>entityClasses.Invitation</code>
Persistence	All SQL. Every statement that takes user input uses a	<code>database.Database</code>

	prepared statement, so input never becomes part of a query string.	
Verification	Semi-automated test drivers in the style of the provided <code>PasswordEvaluationTestingAutomation</code> class, plus a regression suite that runs them all.	<code>testingAutomation</code>

2.2 Architectural decisions made in TP1

Decision	Reasoning	Consequence for the team
Each new Admin function gets its own MVC package rather than more widgets on the Admin Home page.	The Admin Home page was already full, and one package per feature means two feature owners rarely edit the same file.	Four new packages: <code>guiListUsers</code> , <code>guiDeleteUser</code> , <code>guiOneTimePassword</code> , <code>guiManageInvitations</code> . Merges during this phase produced no conflicts in these files.
Policy decisions live in <code>public static</code> controller methods that take their inputs as parameters.	A rule buried inside a button handler cannot be tested without a GUI, so it never gets tested.	<code>checkDeletePolicy</code> , <code>validateInvitationDeadline</code> , <code>parseInvitationDeadline</code> , <code>checkName</code> , <code>checkUserName</code> , <code>checkEmailAddress</code> , and <code>generateOneTimePassword</code> are all exercised directly by the test drivers.
Size is checked before anything else is done with a textual input.	Dropping a very long string into each field is a standard tactic for finding a weakness. The application must refuse it quickly rather than crash or stall.	Every validator returns immediately on oversized input, and every validator has a 10,000-character test case.
The one-time password is stored beside the user's own password, not in place of it.	An Admin issuing a temporary password should not destroy the account's real password; if the user never uses the temporary one, nothing has been lost.	Two new columns, <code>oneTimePassword</code> and <code>oneTimePasswordDeadline</code> , added with <code>ALTER TABLE ... IF NOT EXISTS</code> so an existing database file still opens.
The deadline rules for one-time passwords reuse <code>ControllerAdminHome.validateInvitationDeadline</code> .	Two copies of the same rule drift apart. One copy cannot.	A change to the deadline policy takes effect in both features, and the invitation deadline tests cover both.
The database refuses the last Admin deletion itself, and the page refuses it again before asking.	Defense in depth: the page gives a useful message, and the database guarantees the invariant no matter which page calls it.	Verified twice, at the policy level (test 19) and at the database level (test 17), in <code>UserManagementTestingAutomation</code> .

Figure 3 shows the one-time password story end to end, because it is the story that crosses the most boundaries: an Admin page writes it, the login page consumes it, and the password evaluator decides what may replace it.

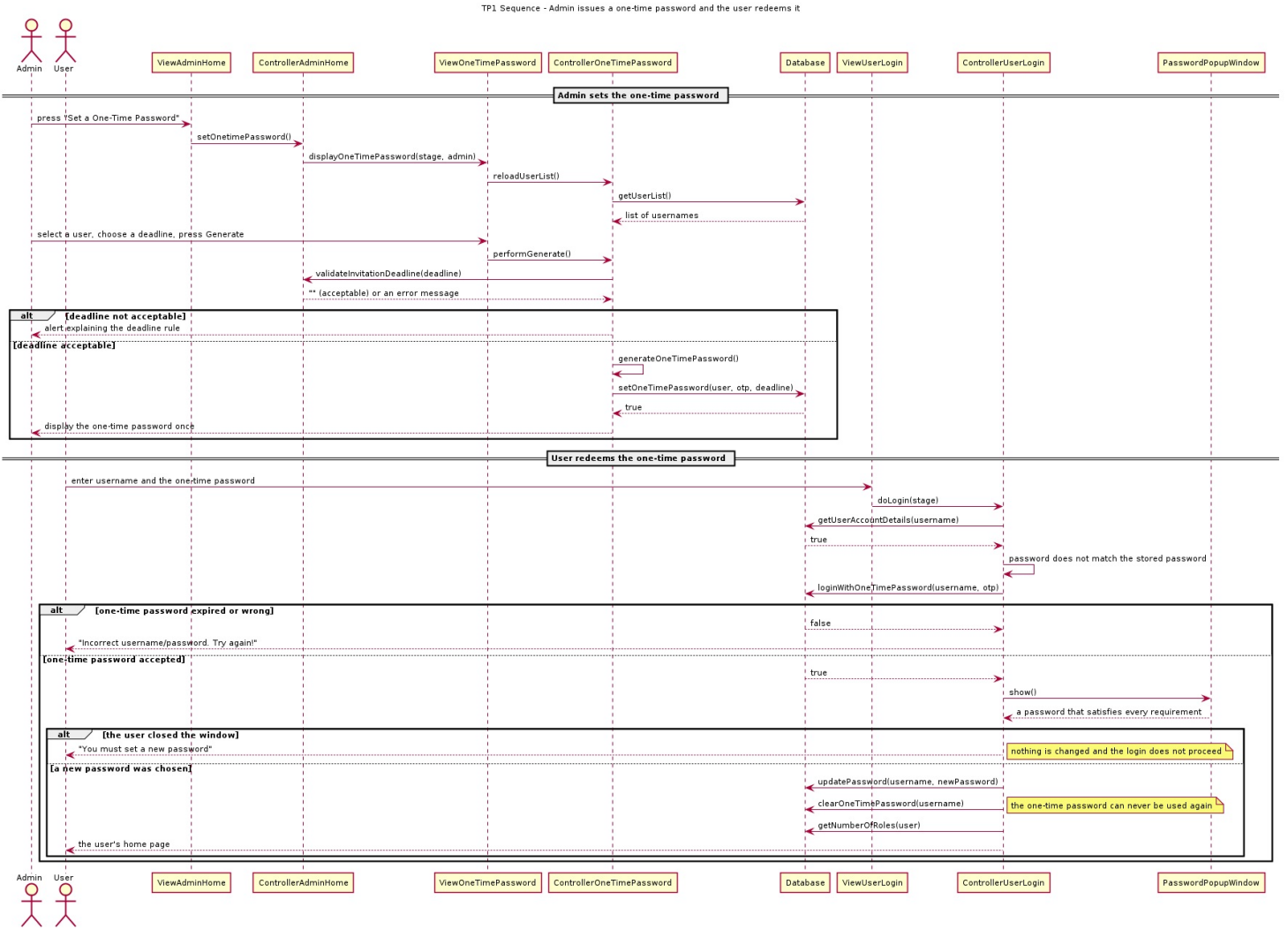


Figure 3 — Sequence: an Admin issues a one-time password and a user redeems it.

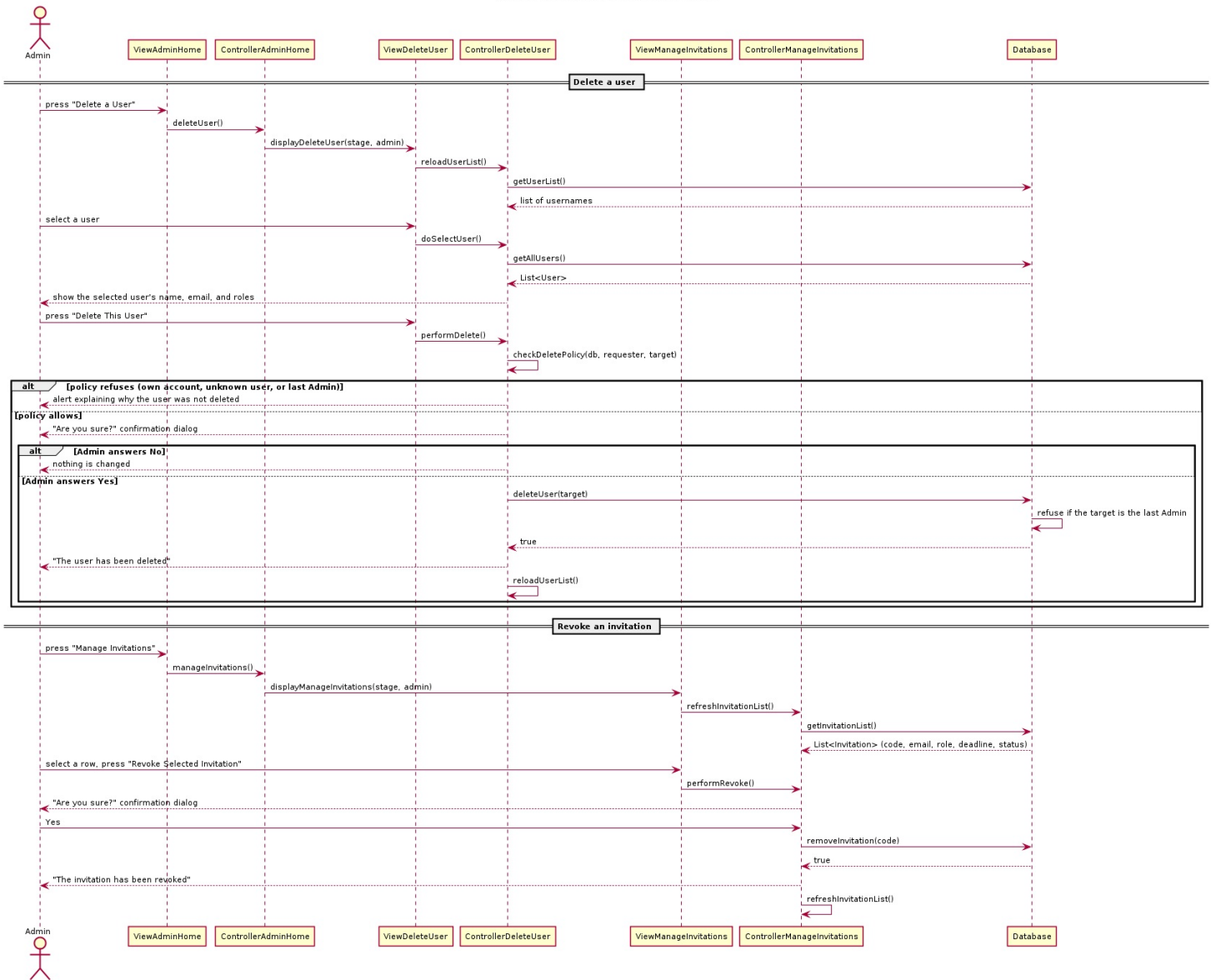


Figure 4 — Sequence: deleting a user, and revoking an outstanding invitation.

4. Input validation design

The provided password evaluator is specified by a directed graph. The email address recognizer added in this phase follows the same pattern, so the two can be read the same way. Figure 5 is its state diagram.

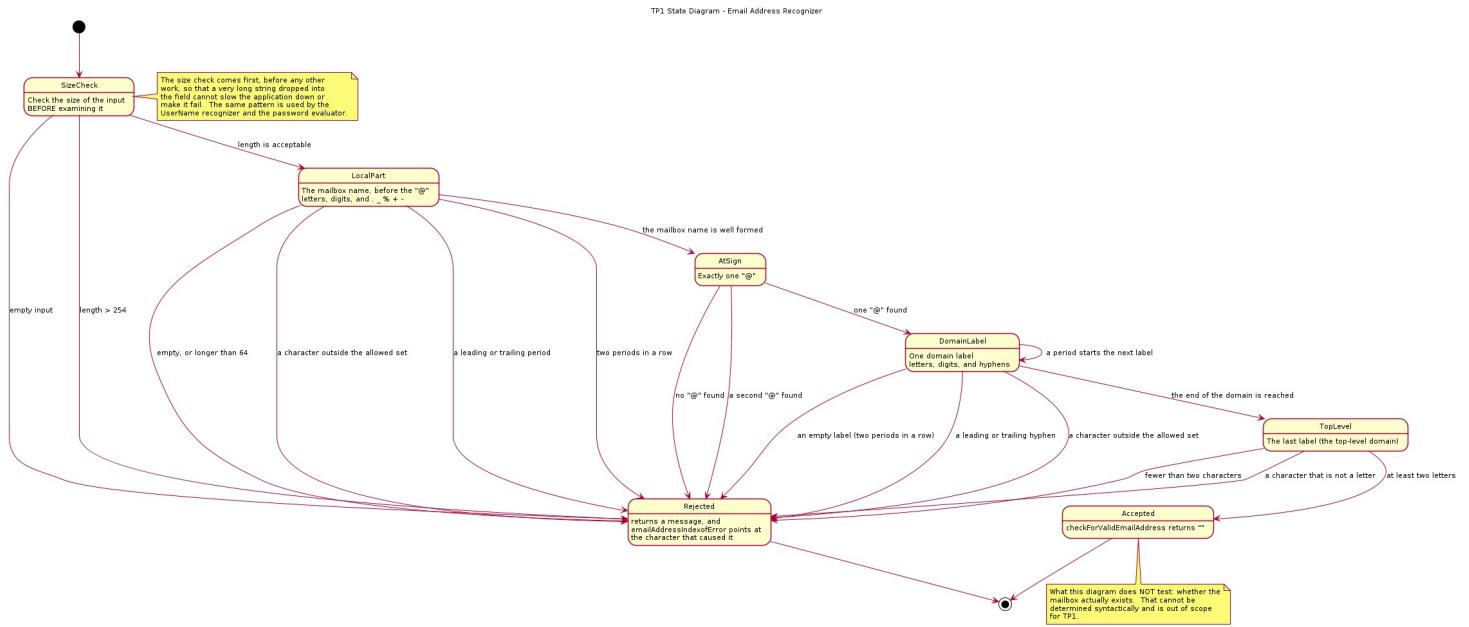


Figure 5 — State diagram for the email address recognizer.

4.1 The requirement the provided password diagram does not test

The provided *Directed Graph — Password Evaluator* diagram tests that the password contains an upper case letter, a lower case letter, a digit, a special character, and that it is long enough. It does not test that the password is *short enough*. Nothing upstream of the diagram tests it either, because the field the user types into accepts any amount of text. The diagram therefore has to be enhanced, and it was: the evaluator now rejects any input longer than `Model.MAX_PASSWORD_LENGTH` (20 characters) before the graph is walked at all, rather than after examining every character of a very long string.

4.2 Why not every input field needs full validation

Every textual field is checked against a size limit, because an oversized string is a probe for a weakness regardless of which field receives it. Beyond that, the depth of validation follows what the value is used for. A value that becomes an identity in the system (a `UserName`), a credential (a password), or a channel the system will act on (an email address) is validated in full, because a malformed value there produces a broken account or an invitation that can never be used. A value that is only stored and displayed back to its owner — a middle name, say — needs a size limit and a character check, but no structural rules, since there is no correct structure for a person's name. And a value the user never types — the role chosen from a `ComboBox`, the date chosen from a `DatePicker` — cannot be malformed, because the widget only offers legal values; the time of day beside the `DatePicker` is typed, so it is parsed and bounded.

What makes the size check non-negotiable everywhere is that it costs one comparison and it removes the easiest way to find out whether the application can be made to fail.

5. Integration and regression

The integration mechanics for this phase, including the merge schedule, the regression checkpoints, and the rule about which diagrams a feature owner updates, are in the companion document *TP1 Integration and Regression Plan*. The short form: no merge is accepted while any test case in `testingAutomation.TP1RegressionSuite` is failing, and the feature owner updates the diagrams their change affects as part of the same merge.